Week 2

To Stimulate CPU scheduling algorithm

1. FCFS
2. SJF

```
#include <stdio.h>
#define MAX 20

void FCFS(int n, int at[], int bt[]) {
    int wt[MAX], tat[MAX];
    int time = 0;
    float total_wt = 0, total_tat = 0;

    for(int i=0; i<n; i++) {
        if(time < at[i]) {          // Check if CPU idle before process arrives
            time = at[i];
        }
        rt[i] = time - at[i];       // response time
        wt[i] = time - at[i];       // waiting time
        time += bt[i];              // execute process
        ct[i] = time;               // completion time of process i
        tat[i] = ct[i] - at[i];     // turnaround time [total time process spends in the system]
        total_wt += wt[i];
        total_tat += tat[i];
        total_rt += rt[i];
    }
    printf("\n --- FCFS --- \n");
    printf("PID \tAT \tBT \tCT \tTAT \tWT \tRT \n");
    for(int i=0; i<n; i++) {
        printf("%d \t%d \t%d \t%d \t%d \t%d \t%d \t%d \n", i+1, at[i], bt[i], ct[i], tat[i],
               wt[i], rt[i]);
    }
    printf("\n Average waiting time = %.2f ", total_wt/n);
    printf("\n Average Turnaround Time = %.2f", total_tat/n);
    printf("\n Average Response Time = %.2f \n", total_rt/n);
}
```

```
void sjf_non_preemptive(int n, int at[], int bt[]) {
    int wt[MAX] = {0}, tat[MAX] = {0}, ct[MAX] = {0},
    rt[MAX] = {0};
    int completed[MAX] = {0}; // tracks completed processes.
    int time = 0, count = 0; // no. of completed processes.
    float total_wt = 0, total_tat = 0, total_rt = 0;
    while (count < n) {
        int min = 9999, idx = -1; // idx = index of selected process
        for (int i = 0; i < n; i++) {
            if (at[i] <= time && completed[i] == 0) {
                if (bt[i] < min) {
                    min = bt[i];
                    idx = i;
                }
            }
        }
        if (idx != -1) { // if process found
            rt[idx] = time - at[idx];
            wt[idx] = rt[idx];
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            total_wt += wt[idx];
            total_tat += tat[idx];
            total_rt += rt[idx];
            completed[idx] = 1;
            count++;
        }
        else {
            time++; // if non found
        }
    }
}
```

```
printf("\n --- SJF Non-preemptive ---\n");
printf("PID \tAT \tBT \tCT \tTAT \tWT \
    \tRT \n");
for(int i=0; i<n ;i++){
printf("%d \t %d \t %d \t %d \t %d \t %d \t %d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i],
    rt[i]);
}
printf("\n Average waiting Time = %.2f", total_wt/n);
printf("\n Average Turnaround Time = %.2f", total_tat/n);
printf("\n Average Response Time = %.2f", total_rt/n);
}

void sjf_preemptive(int n, int at[], int bt[]){
int rt_rem[MAX], wt[MAX] = {0}, tat[MAX]={0};
int ct[MAX] ={0}, rt[MAX];
int first_start[MAX]; first time Process starts (for RT)
for(int i=0;i<n ;i++){
    rt_rem[i] = bt[i];
    first_start[i] = -1;
}
int complete = 0, time =0;
float total_wt=0; total_tat =0, total_rt=0;
while(complete<n){
    int min =9999; idx=-1;
    for(int i=0; i<n; i++){
    if(at[i] <= time && rt_rem[i]>0){
        if(rt_rem[i] < min){
            min = rt_rem[i];
            idx=i;
        }
    }
}
```

```
if (idx == -1) {
    time++;
    continue;
}
if (first_start[idx] == -1)
    first_start[idx] = time;
rt_rem[idx]--;
time++;
if (rt_rem[idx] == 0) {
    complete++;
    ct[idx] = time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];
    rt[idx] = first_start[idx] - at[idx];
    total_wt += wt[idx];
    total_tat += tat[idx];
    total_rt += rt[idx];
}
}
printf("\n --- SJF (preemptive SRTF) ---\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for (int i = 0; i < n; i++)
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}
printf("\n Average waiting time = %.2f", total_wt/n);
printf("\n Average Turnaround Time = %.2f\n", total_tat/n);
}
```

```
int main() {
    int n, choice, at[MAX], bt[MAX];
    printf("Enter number of processes : ");
    scanf("%d", &n);
    for(int i=0; i<n; i++) {
        printf("Enter number of processes : ");
        scanf("%d", &n);
        for(int i=0; i<n; i++) {
            printf("In Process %d \n", i+1);
            printf("Arrival Time : ");
            scanf("%d", &at[i]);
            printf("Burst Time : ");
            scanf("%d", &bt[i]);
        }
        printf("In choose Algorithm:");
        printf("\n 1. FCFS \n 2. SJF - Non Preemptive \n
            3. SJF - Preemptive (SRTF)");
        printf("In Enter choice : ");
        scanf("%d", &choice);
        switch(choice) {
            case 1:
                FCFS(n, at, bt);
                break;
            case 2: sjf_non_preemptive(n, at, bt);
                break;
            case 3: sjf_preemptive(n, at, bt);
                break;
            default:
                printf("Invalid choice");
        }
        return 0;
    }
```

Bafna Gold
Date: Page:

Output

Enter number of processes: 4

process 1

Arrival Time: 0

Burst Time: 7

Process 2

Arrival Time: 8

Burst Time: 3

Process 3

Arrival Time: 3

Burst Time: 4

Process 4

Arrival Time: 5

Burst Time: 6

Choose Algorithm

1. FCFS
2. SJF Non-preemptive.
3. SJF preemptive (SRTF)

Enter choice

FCFS

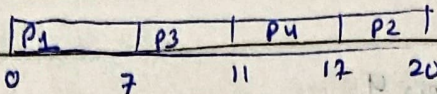
PID	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P3	3	4	11	8	4	4
P4	5	6	17	12	6	6
P2	8	3	20	12	9	9

Average TAT: 9.75

Average wt : 4.75

Average RT : 4.75.

Gantt Chart



Enter choice

--- SJF Non-preemptive ---

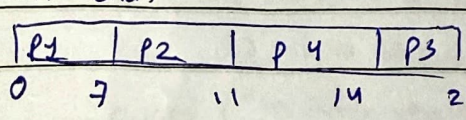
Process	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P2	3	4	11	8	4	4
P3	5	6	20	15	9	9
P4	8	3	14	6	3	3

Avg TAT: 9.00

Avg WT: 4.00

Avg RT: 4.00

Gantt Chart



Enter choice

--- SJF Preemptive ---

Process	AT	BT	CT	TAT	WT	RT
P1	0	7	7	7	0	0
P2	8	3	11	3	0	0
P3	3	4	14	11	7	4
P4	5	6	20	15	9	9

Avg TAT: 9.00

Avg WT: 4.00

Avg RT: 3.25